

DL GEN AI PROJECT REPORT

STUDENT DETAILS:

NAME: Nikhil Itaudiya

ROLL NUMBER: 23f2002113

STUDENT EMAIL: 23f2002113@ds.study.iitm.ac.in

INSTITUTE: IIT MADRAS

PROJECT STATEMENT:

The objective of this project was to build an intelligent system that can solve multiple-choice questions automatically.

The main task was to predict the correct answer for every test question. However, the competition evaluation was based on MAP@3. Therefore, instead of predicting only one answer, the model generated a ranking of the top three most probable answer options.

This is useful because the competition gives partial credit when the correct answer appears in the second or third position.

DATASET UNDERSTANDING AND PREPROCESSING

Dataset Structure

The dataset contains text-based multiple-choice questions. The training data contained 2,000 rows and 8 columns, while the test data contained 500 rows and 7 columns. The main challenge was that the dataset

was relatively small for training large deep learning models.

Duplicate Removal

I checked for duplicate questions using the prompt column.

There were 242 duplicate rows in the training dataset. These duplicate questions were removed to reduce repeated training examples and avoid giving excessive importance to the same question.

After removing duplicates, the dataset contained 1758 unique questions.

Missing Values

I checked for missing values in both the training and test datasets.

No missing values were found in the important columns. As a precaution, missing text values were replaced with empty strings. This prevents errors during tokenization and TF-IDF vectorizations.

Train-Validation Split

The cleaned training dataset was divided into:

80% training dataset and 20% testing dataset

METHODOLOGY

Instead of depending on only one model, I developed and compared four different approaches:

Model 1: TF-IDF + Logistic Regression

Model 2: DistilBERT

Model 3: BERT

Model 4: MLP Neural Network

Finally, I combined the predictions of all four models using a weighted

ensemble.

MODEL ARCHITECTURE AND IMPLEMENTATION

TF-IDF + Logistic Regression

For the first model, I converted every question-option pair into a text input: Question [SEP] Answer Option

For each question, I generated five pairs.

The correct option was assigned label 1 and the remaining four options were assigned label 0.

I used two types of TF-IDF features:

Word-level TF-IDF

This captured words and word combinations using:

n-gram range = (1, 2)

Character-level TF-IDF

This captured character patterns using:

n-gram range = (3, 5)

The word-level and character-level representations were combined.

After this, Logistic Regression was trained to calculate the probability that a question-option pair was correct.

The validation results were:

Metrics	Accuracy	F1-Score	MAP@3
Score	0.9830	0.9825	0.9915

The TF-IDF + Logistic Regression model performed extremely well on the

validation set.

DistilBERT Model

The second approach used the pretrained distilbert-base-uncased model.

For each question, all five options were passed to the model as separate choices.

The transformer generates a score for each of the five options.

The option with the highest score is the predicted answer, while the scores can also be used to generate a ranking.

The model was trained using the AutoModelForMultipleChoice architecture and the AdamW optimizer.

BERT Model

The third approach used the pretrained bert-base-uncased model.

The architecture was similar to DistilBERT, but BERT is a larger transformer model with more parameters.

The input format was: Question + Five Answer Choices

The model directly learned to assign scores to the five possible choices.

The model was fine-tuned using the AdamW optimizer.

MLP Neural Network

I also developed a neural network from scratch.

First, I converted the question-option pairs into TF-IDF vectors with 5,000 features.

Dropout was used to reduce overfitting because the dataset was

relatively small.

Evaluation Metric:

The competition metric was Mean Average Precision at 3.

For every question, the model returns three ranked options.

The final MAP@3 score is the average score across all questions.

The goal was not only to predict the single best answer, but also to produce a good ranking of the top three answers.

Weighted Ensemble

The final system combined the predictions from all four models.

The models produced probability scores for options A to E.

Before combining them, the probability scores were normalized.

The final probability was calculated as:

$$\begin{aligned} \text{Final Score} = & 0.35 \times \text{Logistic Regression Score} \\ & + 0.10 \times \text{MLP Score} + 0.25 \times \text{BERT Score} + 0.30 \times \text{DistilBERT Score} \end{aligned}$$

The five answer options were then sorted according to their final ensemble scores.

The top three options were selected and saved in the required submission format.

This approach was used because different models can make different errors. Combining them can make the final prediction more robust.

Training Details

Weights & Biases was used to track:

#Model configuration

#Validation accuracy

#F1-score

#MAP@3

The trained model weights were later saved and uploaded so that they could be reused during inference.

Error Analysis

The initial simple similarity-based approach used TF-IDF and cosine similarity between the question and each answer option. Its MAP@3 score was only approximately 0.3421.

This showed that simple word similarity was not sufficient for solving MCQs.

The main types of errors identified were:

1. Semantic Similarity Error

An incorrect option may contain similar words to the question and receive a high similarity score.

2. Context Understanding Error

Some questions require understanding the complete meaning of the question and answer rather than matching individual words.

3. Domain Specific Questions

The dataset contains questions from different subjects. A model may

perform well in one domain but struggle in another domain.

4. Less Training Data

There were only around 1,758 unique questions after duplicate removal. This is relatively small for fine-tuning large transformer models.

Key Insights

The most important insight from this project was that a more complex deep learning model is not always automatically better.

The TF-IDF + Logistic Regression model achieved a very strong validation result with a MAP@3 score of 0.9915.

This happened because the dataset was relatively small and the questions had strong textual patterns. The word and character TF-IDF features were able to capture useful patterns efficiently.

Therefore, combining multiple models was a reasonable strategy because:

TF-IDF captures exact words and patterns.

BERT captures contextual meaning.

DistilBERT provides a lighter transformer alternative.

MLP learns nonlinear relationships from numerical text features.

Conclusion

In this project, I developed an end-to-end MCQ solving system.

The final system used a weighted ensemble of four models. The ensemble generated a probability score for each of the five options and returned the top three ranked answers.

The complete deep learning workflow:

understanding the problem → preprocessing data → creating a suitable model input → training models → evaluating with the competition metric → analysing errors → combining models → generating the final submission.

This project helped me understand that solving a real-world machine learning problem is not only about selecting a powerful model. Data quality, evaluation metrics, validation strategy, and error analysis are equally important.